# Mathematical Calculations in YojanaBench

This document catalogue **every mathematical / logical computation performed by the project**, with the exact formula, the source file that implements it, and a worked example. All computations are deterministic Python code — no LLM is ever used for arithmetic, comparison or scoring.

### Notation used below

| Symbol | Meaning |
|---|---|
| `x`, `y` | numeric operands |
| `N` | corpus size (number of indexed clauses) |
| `df(t)` | document frequency of term `t` |
| `tf(t,d)` | term frequency of `t` in document `d` |
| `\|S\|` | cardinality of set / length of list `S` |
| `⌊·⌋` | floor (Python `int()` on a positive float) |
| `mean(S)` | arithmetic mean of the non-null values in `S` |

Constants and tunables are declared in `backend/config.py` and `backend/trace/fraud.py`; they are collected in the summary table at the end.

---

## 1. Tri-state rule engine (eligibility logic)

**File:** `backend/condition_eval.py`

Every eligibility clause is a dictionary `{"field": f, "operator": op, "value": v}` evaluated against the applicant profile. The engine returns a **tri-state** value instead of a boolean:

```
evaluate(applicant, clause) ∈ {True, False, None}
```

- `True` — condition satisfied
- `False` — condition violated
- `None` — field absent/`null` in the applicant (or type mismatch / unknown operator) → drives the `needs_info` verdict

### 1.1 Leaf comparators

| Operator | Definition | Result |
|---|---|---|
| `lt` | `a < b` | bool |
| `lte` | `a ≤ b` | bool |
| `gt` | `a > b` | bool |
| `gte` | `a ≥ b` | bool |
| `eq` | `a = b` | bool |
| `ne` | `a ≠ b` | bool |
| `in` | `a ∈ b` ( `b` is a list) | bool |
| `not_in` | `a ∉ b` | bool |
| `contains` | `b ⊆ a` ( `a` is a container/string) | bool |
| `is_true` / `is_false` | `a = True` / `a = False` | bool |

String operands are normalised before comparison: `actual ← strip(lower(actual))`, `expected ← strip(lower(expected))`, so `"sc" == "SC"` holds. A `TypeError` during comparison (incompatible types) is converted to `None` rather than raising, so an audit never crashes.

### 1.2 Compound conditions

For `{"operator": "and", "clauses": [c₁ … c_k]}` ( Kleene three-valued logic ):

```
AND(c₁…c_k) =  False  if ∃ i : c_i = False
            =  None   else if ∃ i : c_i = None
            =  True   otherwise
```

For `"or"` (dual):

```
OR(c₁…c_k)  =  True   if ∃ i : c_i = True
            =  None   else if ∃ i : c_i = None
            =  False  otherwise
```

### 1.3 Clause-list aggregation

`evaluate_all(applicant, clauses)` evaluates every clause and returns ( `backend/condition_eval.py:93` ):

```
any_false   = ∃ clause c : c.mandatory ∧ evaluate(c) = False
any_unknown = ∃ clause c : c.mandatory ∧ evaluate(c) = None
passed      = ¬any_false ∧ ¬any_unknown
```

### 1.4 Verdict decision (Entitlement agent)

**File:** `backend/agents/entitlement_agent.py:20`

```
eligibility_decision = ineligible  if any_false
                     = needs_info else if any_unknown
                     = eligible    otherwise
```

### 1.5 Independent cross-check (Verification agent)

**File:** `backend/agents/verification_agent.py:18`

A second, separately implemented loop re-derives the decision:

```
independent = ineligible   if ∃ mandatory clause with result False (first hit wins)
            = needs_info   else if ∃ mandatory clause with result None
            = eligible     otherwise

cross_check_agree = (independent = eligibility_decision)   ∈ {True, False}
documents_ok      = (|missing_documents| = 0)
```

The same agent also runs the **document-integrity checks** (§12). Let `integrity_flags` be its output and

```
suspected_forgery = ∃ flag with severity = high
```

### 1.6 Final verdict merge (Orchestrator)

**File:** `backend/agents/orchestrator.py:79` — priority order:

```
decision = ineligible  if eligibility_decision = ineligible
         = needs_info  else if missing_documents ≠ ∅
         = needs_info  else if ¬cross_check_agree
         = needs_info  else if eligibility_decision = needs_info
         = eligible    otherwise
```

The human-approval gate (§4.1) additionally fires when `suspected_forgery` is true, without altering the rule-level `decision` itself.

---

## 2. Entitlement arithmetic (monetary / quantity benefit)

**File:** `backend/agents/entitlement_agent.py`

## 2.1 Rule selection

Entitlement rules are scanned in order; the **first rule** whose guard `when` is absent ( `None` ) or evaluates to `True` is applied. If no rule matches, the entitlement is `0` with the formula string `"no entitlement rule matched the applicant profile"`.

## 2.2 The five compute types

Let `v` = `compute.value`, `u` = `compute.unit`, `m` = `applicant.family_size` (default `m = 1` if absent).

| Type | Formula | Code | Example (dataset) |
|---|---|---|---|
| `fixed_amount` | `E = v` | `_compute_rule` L37 | IGNDPS: `E = 300 INR/month` |
| `per_member` | `E = v · m` | L42 | PDS/PHH: `E = 5 kg × m persons/month` |
| `wage_days` | `E = v · d` where `d` = `compute.days` (default 100) | L49 | MGNREGA: `E = 221 × 100 = 22 100 INR/year` |
| `coverage` | `E = v` (sum insured, not a payout) | L56 | PMJAY: `E = 500 000 INR/family/year` |
| `tiered` | `E = value of the first tier whose guard holds` (see below) | L61 | MUDRA (below) |

Each computation also returns a human-readable **formula string**, e.g. `"5 kg/person/month x 4 members = 20 kg/person/month"`.

## 2.3 Tiered selection

Tiers are ordered lists `{when, value, unit}`. The engine returns the value of the **first** tier whose `when` is `None` (fallback/catch-all) or evaluates `True`; if no tier matches it returns `0`.

### Worked example — MUDRA ( `data/schemes/MUDRA-2023.json` )

| Guard | Tier value |
|---|---|
| `annual_income ≤ 100 000` | 50 000 INR (Shishu) |
| `annual_income ≤ 500 000` | 500 000 INR (Kishore) |
| `when = null` (catch-all) | 1 000 000 INR (Tarun) |

- Applicant income ₹80 000 → tier 1 fires → `E = 50 000`.
- Applicant income ₹300 000 → tier 1 false, tier 2 fires → `E = 500 000`.
- Applicant income ₹800 000 → both guards false, catch-all fires → `E = 1 000 000`.

Because tiers are evaluated in list order, the ordering of the guards is what makes the selection correct — the tiers are written in increasing-income order.

### 2.4 Ablation proxy ("LLM-only maths")

`_compute_naive` (L96) deliberately **skips** all conditional arithmetic and returns the nominal `value` of the *first* rule in the list. The absolute error this produces versus the deterministic computation is the measurable claim of the `llm_math` ablation variant:

```
entitlement_error_abs = |naive_value − reference_value|
```

---

## 3. Hybrid retrieval scoring (RAG agent)

**File:** `backend/retrieval.py`

Each indexed clause is a "document". Two classic IR scorers are computed independently, min–max normalised, then fused linearly.

### 3.1 Tokenisation

`tokenize(text)` = lowercase regex extraction `[a-z0-9%]+` minus a stop-word list ( `the, a, of, to, and, …` ).

### 3.2 BM25 (Okapi)

Parameters $k_1 = 1.5$, $b = 0.75$ ( `retrieval.py:65` ).

$$\text{idf}(t) = \ln\left(\frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} + 1\right)$$

$$\text{BM25}(d, q) = \sum_{t \in q} \text{idf}(t) \cdot \frac{\text{tf}(t,d) \cdot (k_1 + 1)}{\text{tf}(t,d) + k_1 \cdot (1 - b + b \cdot |d| / \text{avgdl})}$$

where $|d|$ is the clause length in tokens and $\text{avgdl} = (1/N) \cdot \sum_d |d|$ .

### 3.3 TF-IDF cosine

$$\text{idf}'(t) = \ln\left(\frac{1 + N}{1 + \text{df}(t)}\right) + 1 \qquad \text{(smooth idf)}$$

$$w(t,d) = \left(\frac{\text{tf}(t,d)}{|d|}\right) \cdot \text{id}'\text{f}(t) \qquad \text{(tf normalised by doc length)}$$

$$\cos(d, q) = \frac{\sum_t q(t) \cdot w(t,d)}{\|q\|_2 \cdot \|d\|_2}$$

with `q(t) = (tf(t,q)/|q|)·idf'(t)` and Euclidean norms `‖v‖₂ = sqrt(Σ_t v(t)²)` (a zero norm is replaced by `1.0` to avoid division by zero).

### 3.4 Min–max normalisation

Both score vectors are mapped to `[0, 1]`:

```
norm(sᵢ) = (sᵢ − min(s)) / (max(s) − min(s))     if max(s) − min(s) ≥ 1e⁻¹²
         = 0                                      otherwise
```

### 3.5 Fusion and ranking

```
combined(d) = w_bm25 · norm(BM25(d)) + w_dense · norm(cos(d))
```

with `w_bm25 = BM25_WEIGHT = 0.5`, `w_dense = DENSE_WEIGHT = 0.5` (`backend/config.py:39`). Results are sorted by `combined` descending and the top `TOP_K = 6` clauses are returned with their rounded score.

---

## 4. Confidence scoring and the human-approval gate

### 4.1 Audit-pipeline confidence (main orchestrator)

**File:** `backend/agents/orchestrator.py:79`

**File:** `backend/agents/orchestrator.py:58` — an additive evidence model:

```
conf ← 0.50                                      (base)
     + 0.20  if |evidence| > 0                    (retrieved clauses exist)
     + 0.15  if cross_check_agree
     + 0.15  if decision = ineligible ∧ ∃ violated mandatory clause
     + 0.10  if decision = eligible ∧ entitlement computed
     + 0.05  if documents were explicitly supplied ∧ documents_ok
     − 0.20  if ¬cross_check_agree

conf ← clamp(conf, 0.05, 0.98),   rounded to 2 decimals
```

**Human-approval gate:**

```
human_approval_required = (conf < CONFIDENCE_THRESHOLD) ∨ ¬cross_check_agree
                        ∨ suspected_forgery                          (§1.5, §12)
CONFIDENCE_THRESHOLD = 0.60        (backend/config.py:43)
```

### 4.2 Money-trace confidence

**File:** `backend/trace/orchestrator.py:73` — starts from a verdict-dependent prior and penalises missing identifiers:

```
conf ← 0.95
conf ← 0.55   if verdict = multiple
conf ← 0.50   if verdict = not_found
conf ← 0.30   if verdict = needs_info
conf ← conf − 0.10   if verdict ∈ {diverted, not_credited} ∧ UTR missing
conf ← clamp(conf, 0.10, 0.99),   rounded to 2 decimals
```

---

## 5. Benchmark evaluation metrics

**File:** `backend/agents/evaluation_agent.py`

Text is normalised before comparison: `normalise(s)` = lowercase, keep only `[a-z0-9]+` tokens, join with single spaces; `tokens(s)` = the same token list.

### 5.1 Exact Match (EM)

```
EM(pred, ref) = 1  if normalise(pred) = normalise(ref)
              = 0  otherwise
```

### 5.2 Token-level F1

With multiset token counts `P = Counter(tokens(pred))`, `R = Counter(tokens(ref))`:

```
common  = Σ_t min(P(t), R(t))                (= |P ∩ R| over multisets)
precision = common / Σ_t P(t)
recall    = common / Σ_t R(t)

F1 = 2 · precision · recall / (precision + recall)
   = 0                          if either side empty or common = 0
```

### 5.3 Verdict accuracy

Only items whose ground truth `verdict ∈ {eligible, ineligible, needs_info}` are scored (informational items such as document/appeal questions are excluded):

```
verdict_correct_i = 1  if predicted_decision_i = ground_verdict_i
                  = 0  otherwise

verdict_accuracy = mean(verdict_correct_i over scorable i)
```

### 5.4 Answer accuracy (lenient)

```
answer_accuracy = ( |{i : F1_i ≥ 0.5}| ) / n
```

### 5.5 Entitlement error

Only when both reference value `r` and predicted value `p` exist:

```
entitlement_error_abs_i = |p_i − r_i|
entitlement_error_rel_i = |p_i − r_i| / |r_i|        (defined only if r_i ≠ 0)

entitlement_error_abs = mean over items with a defined value
entitlement_error_rel = mean over items with a defined value
```

### 5.6 Document F1 (set form)

Only for items with `category = "documents"`; the reference set is the scheme's **mandatory** documents, the prediction is `result.predicted_documents`, both lower-cased/trimmed as sets `P`, `R`:

```
precision = |P ∩ R| / |P|
recall    = |P ∩ R| / |R|
doc_F1    = 2·precision·recall / (precision + recall)     (0 if either set empty or intersection emp

document_f1 = mean(doc_F1 over "documents" items)
```

### 5.7 Expected Calibration Error (ECE)

Confidences are bucketed into `B = 5` equal-width bins over `[0, 1]` (`_calibration`, L177). Bucket index for item `i`:

```
bucket(i) = min( ⌊conf_i · 5⌋ , 4 )
```

Only items with both `confidence` and `verdict_correct` defined are used. With `n = |usable|`, for each non-empty bucket `B_b`:

```
ECE = Σ_b ( |B_b| / n ) · | mean(conf over B_b) − mean(verdict_correct over B_b) |
```

Interpretation: 0 = perfectly calibrated; a value near 0.5 (max possible with 5 bins) means confidence is uninformative.

### 5.8 Aggregation and breakdowns (`aggregate`, L112)

```
n                    = |rows|
exact_match          = mean(EM)
token_f1             = mean(F1)
answer_accuracy      = (count of F1 ≥ 0.5) / n
verdict_accuracy     = mean(verdict_correct) over scorable rows
entitlement_error_abs = mean(abs error)  (rounded to 2 dp)
entitlement_error_rel = mean(rel error)  (rounded to 4 dp)
document_f1          = mean(doc F1)      (rounded to 4 dp)
calibration_ece      = ECE               (rounded to 4 dp)
```

Per-category breakdown repeats EM / F1 / verdict-accuracy means within each `category`. Edge vs. base split:

```
edge_case_accuracy = mean(verdict_correct | edge_case = True)
base_case_accuracy = mean(verdict_correct | edge_case = False)
```

---

## 6. Ablation study deltas

**File:** `backend/ablation.py`

Four configurations are benchmarked: `full`, `no_verification`, `llm_math`, `no_counterfactuals` (defined in `backend/pipeline.py:166`). Each variant's metrics come from §5; the comparison table reports a signed difference against the full pipeline **in percentage points**:

```
Δ(variant) = ( metric_variant − metric_full ) × 100      (rounded to 2 dp)
```

computed on `verdict_accuracy` (`_delta`, L77); `None` propagates when either side is undefined.

---

## 7. Leaderboard ranking

**File:** `backend/leaderboard.py:41`

Entries are sorted by a lexicographic key (best first):

```
rank_key(entry) = ( −verdict_accuracy , −answer_accuracy , +calibration_ece )
```

i.e. higher verdict accuracy wins; ties broken by higher answer accuracy; then by **lower** ECE (better calibration). Missing metrics default to `0` (accuracy-like) or `1` (ECE).

---

## 8. Report aggregation

**File:** `backend/report.py`

```
_acc(values)  = mean of the non-null values          (None if empty)
_mean(values) = mean of all values, rounded to 4 dp  (0.0 if empty)
```

Per-scheme rows are sorted by descending `verdict_accuracy`, with `None` (undefined) pushed to the end: `key = (verdict_accuracy is None, −(verdict_accuracy or 0))`.

Ablation rows are copied from §6.

---

## 9. DBT money-trace mathematics

### 9.1 Reconciliation (sanctioned vs credited)

**File:** `backend/trace/reconcile.py`

```
sanctioned ← txn.sanctioned_amount   (fallback: complaint.expected_amount)
credited   ← txn.credited_amount

shortfall        = round(sanctioned − credited, 2)
credited_ratio   = round(credited / sanctioned, 4)        if sanctioned ≠ 0
partial          = (shortfall is not None) ∧ (shortfall > 0)
amount_disputed  = |expected_amount − sanctioned| > 0.5    (rupee tolerance)
```

Interpretation: `partial = True` ⇒ only part of the money landed; `credited_ratio` = fraction that arrived; `amount_disputed` flags a beneficiary who claims a different sanction than the ledger shows.

**Worked example:** sanctioned ₹2 000, credited ₹1 500 ⇒ `shortfall = 500.00`, `ratio = 0.75`, `partial = True`.

### 9.2 Credit check (did the money reach the beneficiary?)

**File:** `backend/trace/credit_check.py`

Name normalisation: `normalise_name(s)` = lowercase, keep `[a-z]+` tokens, join with spaces (so `"RAVI KUMAR" = "ravi kumar"`).

```
to_expected  = (credited_to_account_last4 = expected_account_last4)   (string equality)
name_matches = (normalise(credited_name) = normalise(beneficiary.name))  or None if either missing
reached      = (credited_amount > 0) ∧ to_expected ∧ (name_matches ≠ False)

status = not_credited  if ledger status = "not_credited" ∨ credited_amount = 0
       = credited      else if to_expected ∧ (name_matches ≠ False)
       = diverted      otherwise
```

Flags: `credit_without_account`, `credited_to_unexpected_account`, `beneficiary_name_mismatch`, `credited_to_business_entity` (the last one if the payee name contains a business word: *enterprises, pvt, ltd, limited, agent, traders, agency, corporate, services, collection*).

### 9.3 Fraud / leakage detection

**File:** `backend/trace/fraud.py`

```
diversion = (credit.status = "diverted")

mule_ring = credited_to_account ∧ ¬to_expected
            ∧ |{distinct beneficiary_id on that account}| ≥ MULE_THRESHOLD
MULE_THRESHOLD = 3

duplicate_utr = |ledger records with this UTR| > 1

severity = high    if mule_ring ∨ diversion
         = medium  else if |flags| > 0
         = none    otherwise
```

`dbt.account_usage(acct)` returns every transaction on the account; the ring is declared when that account serves at least 3 different beneficiaries — the graph-theoretic reading is "receiving account with beneficiary-degree ≥ 3".

### 9.4 Trace verdict (fusion order)

**File:** `backend/trace/orchestrator.py:53` — first matching rule wins:

```
needs_info    if intake invalid
not_found     else if no ledger match
multiple      else if >1 candidate ∧ no UTR given
failed        else if credit.status = not_credited ∧ txn.status = "failed"
not_credited  else if credit.status = not_credited
diverted      else if credit.status = diverted
partial       else if recon.partial
received      otherwise

money_reached_beneficiary = (verdict = "received")
```

Confidence for this verdict follows §4.2.

---

## 10. Counterfactual advisory arithmetic ("how do I qualify?")

**File:** `backend/agents/counterfactual_agent.py`

### 10.1 Smallest satisfying value per leaf clause

`_required_value(clause)` (L63) inverts the comparator — for a violated leaf with threshold `v`:

| Operator | Required value | Direction |
|---|---|---|
| `gte` | `v` | increase |
| `gt` | `v + 1` (numeric) | increase |
| `lte` | `v` | decrease |
| `lt` | `v − 1` (numeric) | decrease |
| `eq` | `v` | set |
| `ne` | `¬v` (bool) / `v + 1` (numeric) / `None` (other) | set |
| `in` | first element of the list | set |
| `is_true` / `is_false` | `True` / `False` | set |
| anything else | `None` | set |

### 10.2 Size of the change

For numeric fields with `current ≠ required`:

```
diff = required − current
verb = reduce  if diff < 0, else increase
amount = |diff|                        (formatted as "Rs. X" for money fields)
```

Special case: for `field = age` with `diff > 0` the suggestion is phrased as "wait until age `required`"; an age *decrease* is marked non-changeable.

### 10.3 Feasibility verification (proof, not guesswork)

All actionable changes are collected into a change set `Δ` (`min_changes = |keys(Δ)|`) and applied to a trial profile; the **same** rule engine re-evaluates:

```
trial       = applicant ⊕ Δ
scenario_ok = ¬any_false(trial)                          (§1.3)
blockers    = suggestions whose field is non-changeable (category, gender, bpl, …)
feasible    = (|blockers| = 0) ∧ scenario_ok
```

### 10.4 Single-change sufficiency

For every actionable suggestion a one-field probe is run:

```
alone_sufficient(s) = ¬any_false( applicant ⊕ {s.field ↦ s.required} )

can_flip_alone = { s.field : alone_sufficient(s) }     (fields that alone flip the verdict)
```

### 10.5 Missing-information accounting

Unknown ( `None` ) clause results contribute the set of leaf fields:

```
missing_info_fields = sorted({ field of every leaf in an unknown mandatory clause })
```

---

## 11. Document-gap arithmetic

**File:** `backend/agents/document_gap_agent.py`

For `k = |missing_documents|` , the i-th missing document (0-based) receives:

```
priority        = i + 1
remaining_after = k − 1
alone_unblocks  = (k = 1)
gate_blocking   = (k > 0) ∧ documents not assumed complete
```

Verdict logic:

```
verdict_if_docs_complete = underlying eligibility decision, except
    underlying = eligible   → "eligible" only after submitting all k documents
    underlying = ineligible → documents cannot change the outcome (stated explicitly)
```

`has_gap = (|unknown_fields| > 0)` when nothing is missing.

---

## 12. Document-integrity checks (fake-paper detection)

**File:** `backend/document_integrity.py`

The checker never reads a document's content. It tests whether every fact an application declares can be true **simultaneously**. Let `a` be the normalised applicant map. Each predicate below emits a flag `{code, severity, detail}` .

**Plausibility predicates** (always evaluated when the field is present):

```
age_invalid            : age ∉ [0, 120]                      (high)
disability_invalid     : disability_percent ∉ [0, 100]        (high)
family_size_invalid    : family_size ∉ [1, 20]               (medium)
income_negative        : annual_income < 0                   (high)
income_implausible     : annual_income > 10,000,000          (medium)
```

**Contradiction predicates** (cross-field consistency):

```
bpl_income_contradiction   : bpl = True  ∧ annual_income > 300,000     (high)
income_tax_contradiction   : tax_payer = True ∧ annual_income < 250,000 (high)
card_bpl_contradiction     : (card ∈ {AAY, PHH} ∧ bpl = False)
                           ∨ (card = APL ∧ bpl = True)                  (high)
widow_age_contradiction    : is_widow ∧ age < 18                       (medium)
education_age_contradiction: level ∈ {UG, PG, Professional} ∧ age < 17 (medium)
secc_pucca_contradiction   : secc_listed ∧ (owns_pucca ∨ house = pucca)(medium)
```

**Evidence predicate** (only when the caller supplied an explicit document list; a QA item with no list stays neutral):

```
unsupported_claim : claim(a) ∧ scheme requires a backing document
                    whose name matches the claim's keywords
                    ∧ no backing document ∈ submitted           (medium)
```

```
suspected_forgery = ∃ flag with severity = high     → raises the human gate (§4.1)
```

In the sandbox an extra flag is enabled:

```
missing_mandatory_document : a required document is absent from submitted (medium)
```

---

## 13. Policy Sandbox arithmetic

**File:** `backend/policy_sandbox.py`

### 13.1 Rule mode — "what if we change a threshold?"

Patching replaces the threshold everywhere the engine reads it, on a deep copy:

```
S' = patch(S, field, v):
        ∀ leaf comparator c in eligibility_clauses,
              entitlement_rules.when, tiers[].when
        where c.field = field ∧ c.operator ∈ {lt,lte,gt,gte,eq,ne}:
              c.value ← v
```

The cohort `C` is the set of unique benchmark applicants for the scheme. For each `a ∈ C`, eligibility and entitlement are recomputed with the *production* engine on `S` and `S'`:

```
newly_eligible   = |{a : dec_after(a) = eligible ∧ dec_before(a) ≠ eligible}|
newly_ineligible = |{a : dec_after(a) ≠ eligible ∧ dec_before(a) = eligible}|

B  = Σ_{a : dec_before(a) = eligible} entitlement_before(a)
B' = Σ_{a : dec_after(a)  = eligible} entitlement_after(a)

budget_delta     = B' − B
budget_delta_pct = 100 · (B' − B) / B            (undefined if B = 0)
```

A transition matrix `T[x→y]` counts every before/after verdict pair; the rows returned are the applicants where the verdict or the amount moved.

### 13.2 Document mode — "what if someone fakes a paper?"

Let $D_0$ be the member's submitted list, or the scheme's full mandatory checklist when unknown. Tampering transforms the pair $(a, D_0)$:

```
omit(doc)   : D' = D_0 \ {doc},   a' = a
forge(doc)  : (a', D') by the mutation table
```

Forge mutation table (keyword match on the document name):

```
income proof        → is_income_tax_payer ← True, annual_income ← 0
ration / BPL card    → bpl ← True, card_type ← APL
age proof            → age ← 150
disability cert      → disability_percent ← 150
any other document   → annual_income ← 10,000,001   (implausible figure)
```

Impact is measured with the checker of §12:

```
detected(a)     = {flag.code in after(a)} \ {flag.code in before(a)}
flags_detected  = Σ_{a ∈ C} 1[code]   per code
affected        = {a : detected(a) ≠ ∅ ∨ verdict/entitlement changed}
```

with the same `newly_eligible`, `budget_delta` and transition accounting as rule mode (forged declarations can change verdicts; an omitted document cannot, so document mode measures detection rather than budget).

---

## 14. Summary of all constants and tunables

| Constant | Value | Meaning | File |
|---|---|---|---|
| `BM25_WEIGHT` | 0.5 | weight of normalised BM25 in the hybrid score | `backend/config.py:39` |
| `DENSE_WEIGHT` | 0.5 | weight of normalised TF-IDF cosine | `backend/config.py:40` |
| `TOP_K` | 6 | number of evidence clauses returned | `backend/config.py:38` |
| $k_1$ (BM25) | 1.5 | term-frequency saturation | `backend/retrieval.py:65` |
| `b` (BM25) | 0.75 | length normalisation | `backend/retrieval.py:65` |
| `CONFIDENCE_THRESHOLD` | 0.60 | human-approval gate | `backend/config.py:43` |
| confidence clamp (audit) | [0.05, 0.98] | bounds of the additive model | `backend/agents/orchestrator.py:75` |
| confidence base / penalties | 0.50, +0.20, +0.15, +0.15, +0.10, +0.05, −0.20 | additive evidence model | `backend/agents/orchestrator.py:58` |
| trace confidence base | 0.95 (0.55 / 0.50 / 0.30 per verdict, −0.10 missing UTR) | money-trace confidence | `backend/trace/orchestrator.py:73` |
| trace confidence clamp | [0.10, 0.99] | bounds | `backend/trace/orchestrator.py:82` |
| `MAX_PLAUSIBLE_AGE` | 120 | integrity: impossible age bound | `backend/document_integrity.py` |
| `MAX_DISABILITY_PERCENT` | 100 | integrity: impossible disability bound | `backend/document_integrity.py` |
| `MIN/MAX_FAMILY_SIZE` | 1 / 20 | integrity: implausible family size | `backend/document_integrity.py` |

| Constant | Value | Meaning | File |
|---|---|---|---|
| `BPL_INCOME_LIMIT` | ₹3,00,000 | BPL vs income contradiction | `backend/document_integrity.py` |
| `TAX_PAYER_MIN_INCOME` | ₹2,50,000 | income-tax floor contradiction | `backend/document_integrity.py` |
| `IMPLAUSIBLE_INCOME` | ₹1,00,00,000 | doctored-figure threshold | `backend/document_integrity.py` |
| `MIN_COLLEGE_AGE` | 17 | UG/PG vs age contradiction | `backend/document_integrity.py` |
| ECE buckets | 5 | calibration bins | `backend/agents/evaluation_agent.py:177` |
| answer-accuracy F1 cut | 0.5 | lenient answer metric | `backend/agents/evaluation_agent.py:118` |
| entitlement dispute tolerance | ₹0.5 | ledger vs complainant amount | `backend/trace/reconcile.py:29` |
| `MULE_THRESHOLD` | 3 | distinct beneficiaries per receiving account | `backend/trace/fraud.py:16` |
| ablation delta scaling | ×100 (pp) | percentage-point reporting | `backend/ablation.py:81` |
| default `family_size` | 1 | fallback for `per_member` | `backend/agents/entitlement_agent.py:43` |
| default `wage_days` | 100 days | MGNREGA-style default | `backend/agents/entitlement_agent.py:50` |

## 15. Where the maths is verified

The formulas above are exercised by the test suite:

```
python -m pytest -q
```

Relevant tests: `tests/test_agents.py` (rule engine, entitlement arithmetic, confidence), `tests/test_advisors.py` (counterfactual verification), `tests/test_ablation.py` (variant deltas), `tests/test_grievance.py` (reconciliation, fraud thresholds), `tests/test_pipeline.py` (end-to-end metrics), `tests/test_document_integrity.py` (§12 predicates, tamper transforms), `tests/test_policy_sandbox.py` (§13 rule and document modes).

Two documented ablation claims follow directly from this maths:

1. **Deterministic entitlement maths beats naive "headline number" grabbing** — measured by `entitlement_error_abs` under `full` vs `llm_math` (§2.4, §5.5).
2. **The verification cross-check catches disagreement** — measured by `verdict_accuracy` under `full` vs `no_verification` (§1.5, §6).